

Min-Conflicts Local Search on a Terrain Map-Coloring CSP

Jacob A. Geldbach
College of Engineering: CS
University of Idaho
 Moscow Idaho, United States
 geld8788@vandals.uidaho.edu

Abstract—Min-Conflicts is a local search algorithm useful for solving constraint satisfaction problems (CSPs). In this project, Min-Conflicts was applied to a terrain detailed grid map coloring CSP, where different terrains acted as the variables and the constraints were related to which terrains could be adjacent to each other. The algorithm was tested on 5 different map sizes, with and without diagonal neighbors.

Index Terms—Local Search, Constraint Satisfaction Problem, Min-Conflicts, Map Coloring, Convergence

I. INTRODUCTION

In this project, the Min-Conflicts algorithm was successfully implemented in order to solve a terrain map-coloring CSP problem. The terrains were represented as a grid of cells which is generated randomly and represented in real time as the algorithm runs. The Python process uses the Pygame visualization library to display the initial random terrain assignment for the inputted map size, then continues to display an update of the grid every 50 steps until either the algorithm finds a solution or the stopping criterion is met. The constraints for this problem are the adjacency rules for the terrains defined below in Table ???. This project's Python process has the ability to run headless without generating the visualization at all, this was useful for the data gathering portion of the project where I ran 16 trials for each map size. The ability to run the algorithm headless allowed for the trials to be ran in parallel and gather the data much faster as on the larger map sizes the Min-Conflicts algorithm took a long time to either converge or reach the stopping criterion, the reason for this is discussed in the sections below. The algorithm was successful in converging on a solution in all 10 conditions tested, and the data gathered allowed for a clear representation of how this CSPs time complexity scales with the size of the map or the number of variables constrained.

TABLE I
TERRAIN ADJACENCY COMPATIBILITY MATRIX

Terrain	Water	Beach	Lowland	Forest	Hills	Mountains
Water	✓	✓	×	×	×	×
Beach	✓	✓	✓	×	×	×
Lowland	×	✓	✓	✓	×	×
Forest	×	×	✓	✓	✓	×
Hills	×	×	×	✓	✓	✓
Mountains	×	×	×	×	✓	✓

A ✓ marks a legal adjacency (elevation differs by at most one step); × marks a constraint violation. Every terrain is trivially compatible with itself.

II. ALGORITHM DESCRIPTION

Min-Conflicts being a local search algorithm starts in a problem search space by randomly choosing a potential solution, in this case that is a complete assignment of values to the variables. The algorithm must then "grade" or determine how many constraint violations this grid state has, this is where the majority of the time complexity comes from. The algorithm must first iterate over each cell in the grid and check each of its neighbors to see if any of the constraints are violated, each cell with a constraint violation is then added to a list of conflicted cells. A conflicted cell is then chosen at random and the algorithm iterates over all the possible values for that cell and determines which value would improve the overall conflicted state of the current grid state. These two steps result in time complexities described by $O(n * d)$ and $O(k * d)$ respectively, where n is the number of cells or variables, d is the number of neighbors for each cell, and k is the number of possible values for each variable or terrains each cell can be colored. Due to in this problem k being a constant value of 5, and d being a constant value of 4 or 8, the time complexity for the step of finding a new terrain for the chosen conflicted cell is negligible and considered constant $O(1)$. Min-Conflicts being a local search algorithm benefits from the memory efficiency of only storing the current state of the grid and that of the candidate state being tested, therefore the space complexity is $O(n)$ where n is the number of cells on the grid. Min-Conflicts is not guaranteed to find a solution if one exists, this is due to the possibility of plateaus or local maximum in the search space where the algorithm

can get stuck and not be able to find a better solution. Due to this fact a stopping criterion is essential to prevent the algorithm from running indefinitely. In this project I chose a stopping criterion of steps since improvement. Essentially if a certain number of iterations or steps have passed without the algorithm finding a better solution, i.e. decreasing the total number of conflicts, then the algorithm will stop and report a failure to converge. Due to the number of variable scaling of Min-Conflicts time complexity, I also have the stopping criterion of steps since improvement scaling with the size of the grid, where $steps_since_improvement = w * h$ where w and h are the width and height of the grid. I chose this so that the algorithm has a fair chance to converge on larger variable conditions and so that the algorithm could stop when in a clear plateau or local maximum instead of continuing on for tens of thousands of more iterations until a static maximum number of steps criterion was hit. A key benefit of Min-Conflicts shows itself in this problem space especially in the larger map size conditions, and it lies in the first step of starting from a random state of values assigned to the variables. By starting in this random potential solution, the exponential time complexity of building a solution to the CSP incrementally (really finding the solution) by traversing a tree of potential value assignments is able to be avoided. Restated, although this approach is complete, to incrementally build a grid of terrains that do not have any conflicts would be of time and space complexity of $O(k^n)$ where n is the total number of cells in the grid and k is the 6 terrain choices. Take the 200x200 largest grid condition, where $n = 40,000$, to incrementally find the solution the time and space complexities are $O(6^{40,000})$, practically INF in terms of even modern day computers. In other words, Min-Conflicts, by sacrificing completeness, allows for that condition in practice to become solvable. In this project, a greedy flag or non-classic Min-Conflicts argument was added which would instruct the algorithm to pick the next conflicted cell from a list of the *most* conflicted cells (i.e. cells were sorted by the number of neighbors they were conflicting with) instead of picking *any* conflicted cell. The thought process behind this addition was that by tackling the most conflicted cells first it may speed up time or lessen the number of steps before convergence. Counter-intuitively, this caused the algorithm to plateau more often, which will be discussed in more depth in later sections.

III. RESULTS

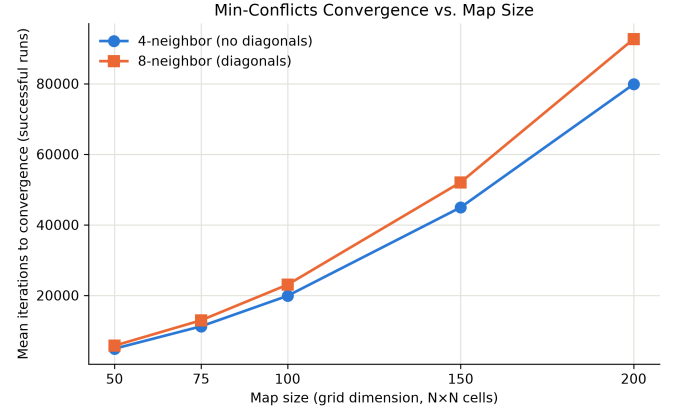


Fig. 2. Graph of the difference graph sizes vs average iterations until convergence for successful runs

IV. DISCUSSION

After seeing the results of the 160 runs in total across the 10 different conditions, we can confirm that the time complexity, measured here in iterations, roughly did scale linearly based on n . This aligns with the theoretical time complexity discussed in the earlier Algorithm section. Looking at each row of Table II, the Avg. Iterations column scales at the same rate as n number of grid cells scales. Ex: 50x50 or 2500 cells took 5000 average iterations to succeed and the 100x100 grid with 10000 cells or $4n$ of that of the 50x50 grid took 4 times the average number of iterations at around 20000. One of the most surprising results from the data was the 100% successful convergence rate for all of the diagonal 8 neighbor conditions. That is the opposite of what I expected when beginning this experiment. When looking back at the poor results from the greedy most conflicted cells grouping experiment, the additional argument added in this project that essentially limited the pool of cells to randomly pick the next cell to change from, this makes more sense in retrospect. With 8 neighbors, or twice the number of potential conflicts, this allows for a 'stronger' choice to be made in what terrain to switch to, now there is the possibility for up to 8 conflicts to be improved on instead of just 4. Also when just taking into account the 4 neighbors above below left and right, this could lead to a checkerboard layout occurring which seemed to lend itself into more oscillation occurring on the grid, switch back and forth making little to no progress over multiple iterations. This oscillation was also the cause of the majority of runs that would stall out and fail, going back and forth on terrain changes on the same cells until the steps without improvement criterion was hit. Other failures were due to just simply getting stuck in grid states where, no matter what random cell was chosen from the conflicted list, as many conflicts were created as were solved every iteration. The only way for this algorithm to get past those states is with random restarts, which are essentially just another new run of the algorithm. When looking at the after visualizations

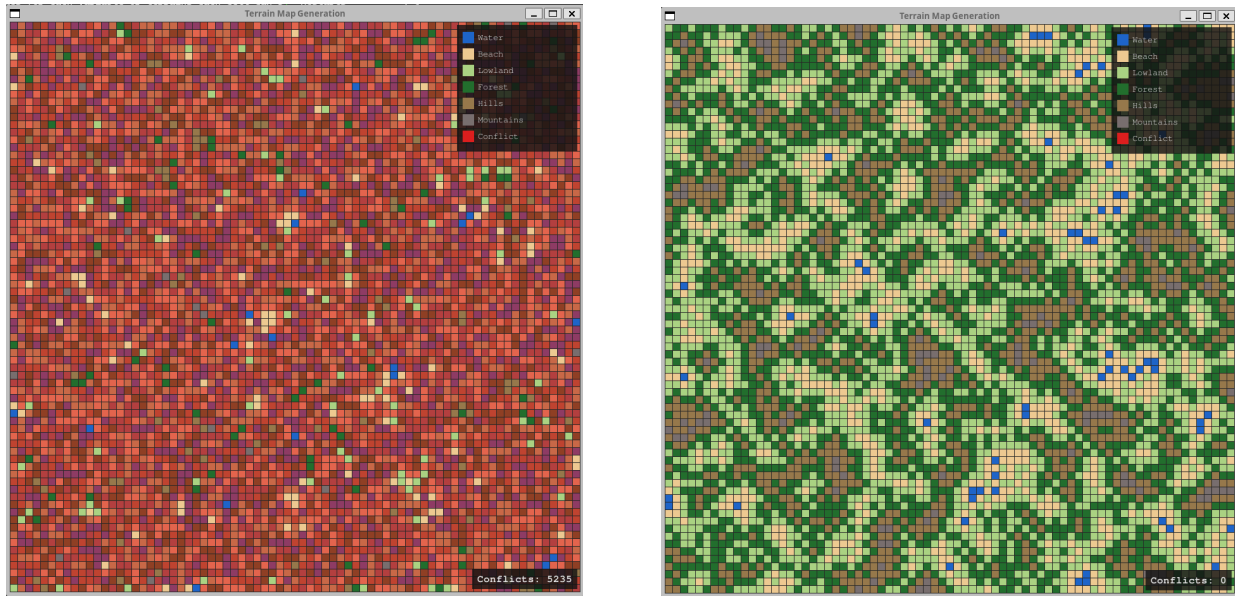


Fig. 1. Left: Beginning random terrain assignments for the 75x75 grid size Right: Successful convergence of a solution

TABLE II
MIN-CONFLICTS — CONVERGENCE RESULTS (16 TRIALS PER CONDITION)

Map Size	Diagonals	Avg. Iterations (success)	Failures (out of 16 trials)	Stopping Criterion
50×50	Off	4892	2	2500 steps without improvement
50×50	On	5752	0	2500 steps without improvement
75×75	Off	11225	3	5625 steps without improvement
75×75	On	12939	0	5625 steps without improvement
100×100	Off	19877	3	10000 steps without improvement
100×100	On	23038	0	10000 steps without improvement
150×150	Off	44937	4	22500 steps without improvement
150×150	On	52045	0	22500 steps without improvement
200×200	Off	79852	12	40000 steps without improvement
200×200	On	92672	0	40000 steps without improvement

of Figure 1 it is easy to see that this algorithm focuses on building a local non-conflicting solution surrounding the single cell that was chosen for that iteration, because of that the grid looks like a lot of different local solutions glued together when it is finished converging on a solution. It does not look like a naturally occurring geographical biome, as this is not a globally scoped algorithm where it can take the entire grid into consideration every single iteration. That is the cost associated with the ability to actually find a solution to this CSP in practice, talked about briefly in the Algorithm Section. That is also the key difference in this project compared to Project 1 [?]. Min-Conflicts is a local search algorithm and all the search algorithms discussed in Project 1 were global search algorithms. Each one incrementally builds the solution starting from the same state, and with that comes the requirement of remembering past cell values and traversing many layers of the tree representing the cells' relation to each other. We have already discussed how this is not practical or even possible with some of the larger grid sizes in this project, making Min-

Conflicts and local search the better choice.

USE OF AI

Coding

The Claude Sonnet 5.0 Model was used for the initial implementation of the project's argument parsing, CSP/terrain constraint logic, and the Pygame visual library set up of the realtime terrain grid. This was done by providing the model with a detailed root level .md file. Other than that, the majority of the core Min-Conflicts algorithm and its stopping criteria were written by hand.

Writeup

The Claude Sonnet 5.0 Model was used for all the Latex structural code and library inclusions needed for this document. The content of each section of the document was written by hand and are my own words. The model was used to help with the formatting of the tables and figures, but the data content of the tables were gathered manually by running the

program and recording the data. The model was also used for grammar and spelling checks of the section content.

REFERENCES

- [1] J. A. Geldbach, "Analysis of Search Algorithms on Cost Grid-Maps," unpublished, CS 5701, University of Idaho.